

02 — Triage & Collection

Velociraptor and KAPE for scalable DFIR acquisition

Educational / Defensive DFIR Study Notes

LEARNING OBJECTIVES

- Choose Velociraptor vs KAPE for live hunt vs targeted disk triage
- Explain KAPE Target vs Module and name key collectors/parsers
- List high-value Windows artifacts to collect first
- Preserve forensic soundness: order of volatility and documentation

SCOPE REMINDER

- This material covers detection, artifacts, and incident response only.
- Attacker actions are phrased as 'adversaries may...' then pivot to evidence.
- Tool/command names appear as IOC strings / exam knowledge — not how-to guides.

Triage mindset

Triage is not full forensics — it is **enough evidence, fast enough** to decide containment and next deep-dives. Over-collection wastes time; under-collection loses volatile proof. Prefer: memory (if safe) → critical logs/registry/MFT → broader disk.

Velociraptor

Velociraptor is an open-source endpoint visibility and DFIR platform. For exams, remember four verbs: **collect, hunt, live response, artifacts**.

Capability	What it means	When to use
Collection	Pull defined artifact sets from endpoints to the server	Known IOCs; scoped incident hosts
Hunt	Query many endpoints for a condition (file hash, mutex, reg key)	Fleet-wide: 'who has this beacon path?'
Live response	Interactive investigation on a live host via VQL	Hot box where reboot would destroy evidence
Artifacts	Reusable VQL packs (Windows.EventLogs, prefetch, etc.)	Standardize what juniors collect

Velociraptor shines when you need **scale** and **repeatable hunts**. Pair with offline parsers (or export) when deep timeline work is needed on a workstation.

KAPE — Eric Zimmerman's triage kit

KAPE (Kroll Artifact Parser and Extractor) separates **what to gather** from **how to parse**:

- **Targets** — collection definitions (copy/compound sources from disk)
- **Modules** — post-processing tools run against collected data

High-yield Targets (collect)

Target area	Why it matters	Typical contents
MFT (\$MFT)	Filesystem timeline; deleted file metadata remnants	Record of create/modify/access/entry; parent paths
Event Logs	Execution, logon, service, PowerShell, Sysmon	Security, System, Application, Microsoft-Windows-*
Registry	Persistence, user activity, system config	SYSTEM, SOFTWARE, SAM, NTUSER.DAT, UsrClass.dat
Prefetch	Evidence of program execution (Windows client)	*.pf with run count and last run times
Amcache	Program inventory / execution-related metadata	Amcache.hve — full paths, SHA1 (versions vary)

High-yield Modules (parse)

Module / tool	Input	Output value
MFTECmd	\$MFT (and related)	CSV/bodyfile timelines; find staging paths, deleted names
EvtxECmd	Windows EVTX	Normalized CSV; easier hunting across Event IDs
RECmd / Registry Explorer	Registry hives	Batch queries for Run keys, services, userassist, etc.

Suggested collection order (practical)

- Document: hostname, time sync, who authorized collection, hash of images/exports
- If memory in scope: capture RAM before heavy disk thrashing when possible
- KAPE Targets for MFT + EVTX + Registry + Prefetch + Amcache on impacted Windows hosts
- Run Modules; feed Hayabusa / timelines; escalate interesting hosts to full disk/memory
- Velociraptor hunt across estate for the same IOCs discovered in triage

How do I prove X?

- **Program ran:** Prefetch + Amcache + 4688/Sysmon 1 + UserAssist / Bam — prefer corroboration
- **File existed then deleted:** MFT + USN Journal via MFTECmd workflow; \$LogFile if available
- **Fleet still infected:** Velociraptor hunt for hash/path/mutex; do not assume one host = all

Common exam traps

- Confusing Target (collect) with Module (parse)
- Skipping Prefetch on servers where Prefetch may be disabled — know the OS role
- Collecting only Security.evtx and missing Sysmon / PowerShell / Task Scheduler logs
- Forgetting chain of custody and hashing of collected archives
- Running heavy Module parsing on the victim (prefer collect → analyze offline)

QUICK REFERENCE

- Velociraptor: collect | hunt | live response | artifacts
- KAPE Target = collect (MFT, EVTX, Registry, Prefetch, Amcache)
- KAPE Module = parse (MFTECmd, EvtxECmd, RECcmd)
- Order of volatility: memory → logs/registry → full disk
- Triage = decide next steps, not write the full report yet

Mini Self-Check

Answer these without looking back. Answers are on the next page.

- Q1.** Differentiate a KAPE Target from a KAPE Module with one example each.
- Q2.** Name five high-value Windows collection areas commonly targeted by KAPE.
- Q3.** Which Velociraptor capability finds a known malicious mutex across 500 hosts?
- Q4.** Why parse EVTX offline with EvtxECmd after KAPE collection?
- Q5.** What should you document before/during collection for court-ready work?

Self-Check Answers

- A1. Target collects (e.g., Event Logs / MFT). Module parses (e.g., EvtxECmd / MFTECmd).*
- A2. MFT, Event Logs, Registry, Prefetch, Amcache.*
- A3. Hunt (artifact/VQL hunt for that mutex).*
- A4. Normalize and search at scale without altering the victim; faster multi-ID analysis; preserves originals.*
- A5. Authorization, time/timezone, hostname, collector identity, tool versions, hashes of exports, chain of custody.*